VideoComposer: Making Motion, Not Just Appearance, Controllable in Video Generation

A NeurIPS 2023 paper from Alibaba Group and Ant Group treats a video as a composition of textual, spatial, and temporal conditions, and turns motion vectors from compressed video into an explicit control signal.

Controllable image generation has come a long way. Hand a diffusion model a sketch, a depth map, or a prompt, and it will return an image whose structure, layout, and style are broadly what you asked for. Move the same idea to video, though, and it stops working cleanly. Video carries a temporal axis that images don't: the way things move varies enormously from clip to clip, and every generated frame has to stay consistent with its neighbors instead of drifting on its own. The spatial control signals designed for a single image simply don't tell the model how anything should move.

Put differently, image generation answers "what to draw," while controllable video generation also has to answer "how it should move." VideoComposer, from Alibaba Group and Ant Group and published at NeurIPS 2023, is aimed squarely at that second question. Its central claim is that to control a video you have to control its content and its motion together, not just its appearance.

To do that, the paper decomposes a video into three families of freely combinable conditions: textual, spatial, and temporal. A user supplies any subset, and the model recomposes a video that obeys all of them at once. That is the "compositional" in the title. The single most distinctive move is to reuse the motion vectors already sitting inside compressed video as an explicit temporal control signal. The rest of this post unpacks how those conditions are encoded and fused, and why motion vectors turn out to be such a convenient handle on time.

Paper: https://arxiv.org/abs/2306.02018 · Code: https://github.com/damo-vilab/videocomposer · Project page: https://videocomposer.github.io

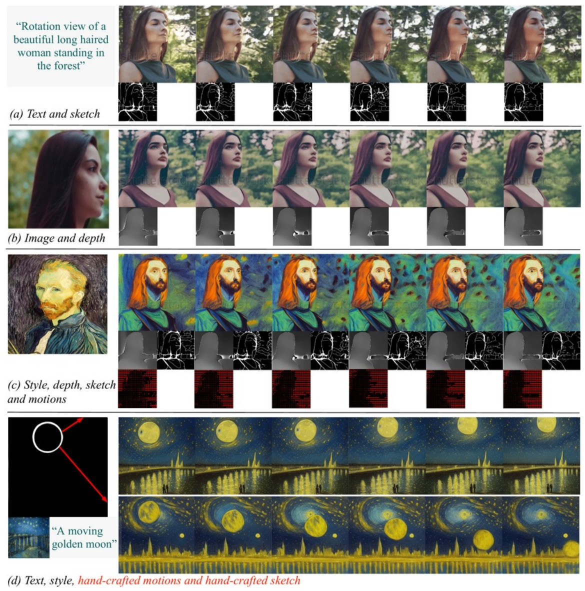

Figure 1. Compositional video synthesis at a glance. (a-c) VideoComposer generates videos that follow textual, spatial, and temporal conditions, or any subset of them; (d) from just two hand-drawn strokes, a red one specifying the motion and a white one specifying the shape, it synthesizes a video that matches the intended movement and form. The figure sets up the core intuition: conditions snap together freely, and motion itself becomes something the user can edit.

The hard part of controllable video is motion

Diffusion models have made customizable image generation remarkably controllable, and the obvious next step is to extend that control to video. Video adds two difficulties images never had. The first is the sheer variety of temporal dynamics: the same caption, "a tiger walking," spans a huge space of speeds and directions. The second is inter-frame consistency: each generated frame must stay coherent with the frames around it, or the result flickers and jumps.

Prior customizable-image methods mostly offer spatial control. They can pin down what a scene looks like, but they give no principled handle on how it evolves over time. Stacking single-frame controls like sketches or depth maps frame by frame tends to break consistency across frames. Controllable video synthesis stayed an open problem precisely because it lacked an interface that could describe motion explicitly without sacrificing consistency.

The method: a video as a composition of conditions

VideoComposer is built on a Video Latent Diffusion Model (VLDM), which denoises video in a compressed latent space for efficiency. During training, a video is decomposed into three kinds of conditions: textual, spatial (a single image or a single sketch), and temporal (motion vectors, depth sequences, mask sequences, sketch sequences). Because the conditions are freely combinable, the model learns to reassemble a video that satisfies whatever subset it is handed.

The architecture below is the key to the whole paper. Three paths are worth tracing: how every sequential condition flows into one shared encoder, how text and style are injected separately, and how the encoded conditions join the noisy latent inside the diffusion backbone.

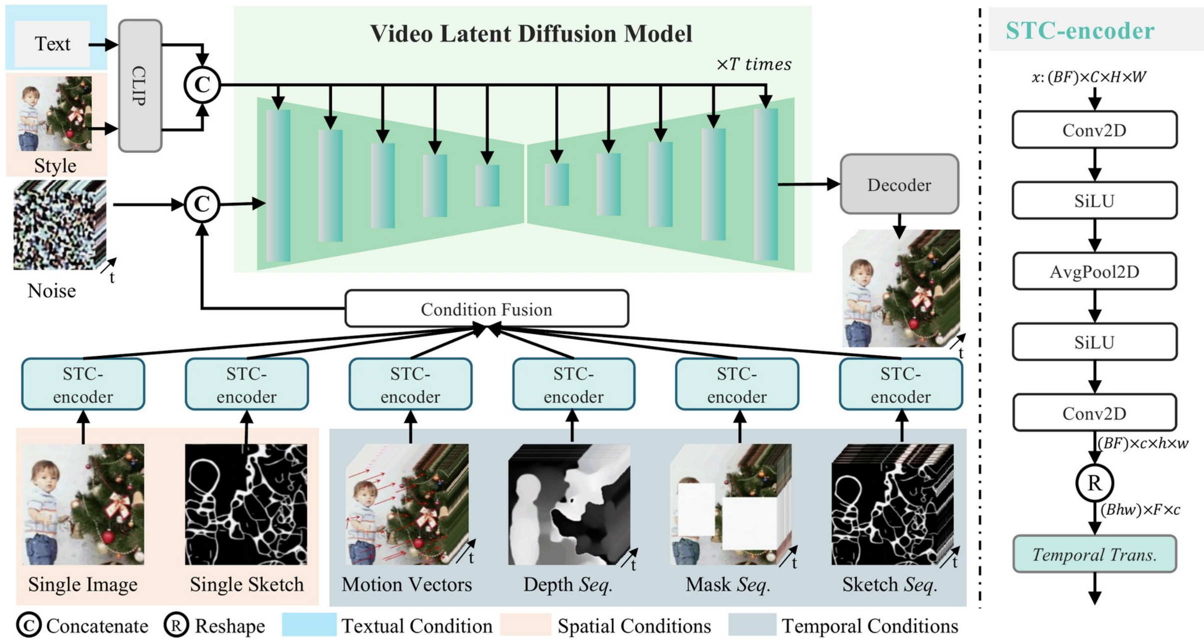

Figure 2. Overall architecture of VideoComposer. Left: a video is decomposed into textual, spatial, and temporal conditions; the sequential ones are all embedded by a shared STC-encoder, while text and style are injected via CLIP. Middle: the encoded conditions are summed in Condition Fusion, concatenated with the noisy latent along the channel dimension, and used to jointly guide the VLDM's denoising before a decoder reconstructs the video. Right: inside the STC-encoder, two Conv2D layers plus pooling capture local spatial structure, and a temporal Transformer models change across frames. This module is what unifies heterogeneous conditions while improving temporal consistency.

Three design choices are worth holding onto. First, a unified interface: motion vectors, depth maps, masks, and sketch sequences all pass through the same Spatio-Temporal Condition encoder (STC-encoder), where two convolutions and a pooling layer capture local spatial structure and a temporal Transformer models how things change across frames. That both accommodates heterogeneous conditions and, by modeling time explicitly, improves inter-frame consistency. Second, fusion: the encoded conditions are summed element-wise and concatenated with the noisy latent along the channel dimension before entering the diffusion network, while text and style enter through cross-attention. Third, a two-stage schedule: the model is first pre-trained on plain text-to-video, then trained compositionally, with DDIM sampling and classifier-free guidance at inference.

The denoising objective is the standard diffusion loss: the network learns to predict the added noise from the noisy latent, given the conditions and the timestep,

$L_{VLDM} = E[\, \| \epsilon - \epsilon_\theta(z_t, c, t) \|^2 \,]$, where c is the freely combinable set of conditions above.

The key move: motion vectors as an explicit control signal

The most recognizable idea in VideoComposer is finding a ready-made, cheap, and explicit representation of motion: motion vectors. They already exist inside compressed formats like H.264, where they encode the displacement between adjacent frames. The paper extracts them and treats them as one temporal condition, which means where things move and how fast is no longer left to the model but is something the user can prescribe directly.

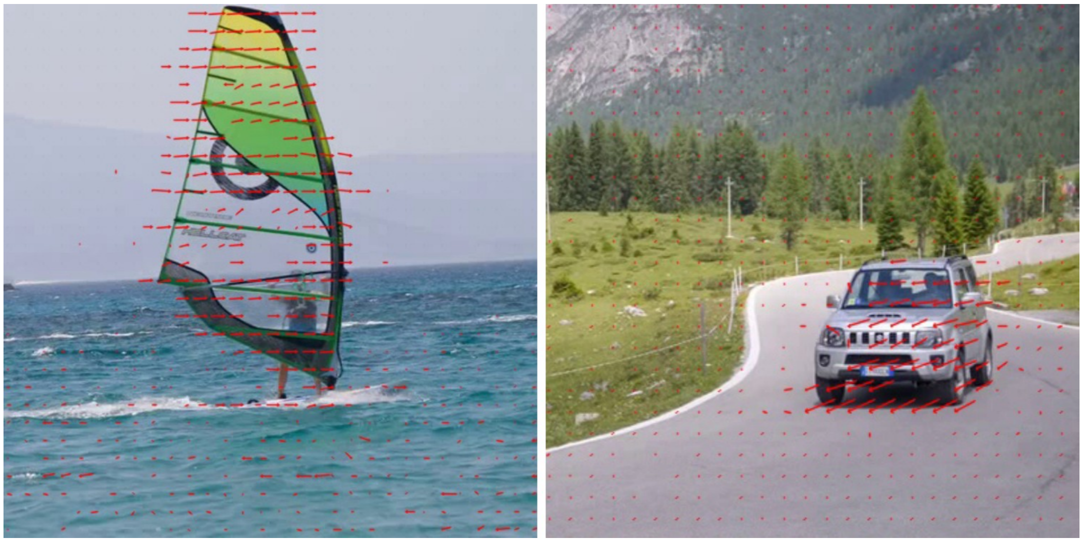

Figure 3. Examples of motion vectors. Small red arrows overlaid on the source video encode the direction and magnitude of local motion: the windsurfer's sail lifts upward while the water spreads outward, and the off-road car moves down and to the right as a whole. Notice how the motion vectors strip away most static appearance and keep only where and how things move, which is exactly what makes them a good temporal control signal.

Results: no cost to text-to-video, finer conditional control

A natural worry is whether piling on all these compositional conditions hurts basic text-to-video quality. The paper answers on the MSR-VTT benchmark. In the zero-shot setting, VideoComposer reaches an FVD of 580 and a CLIPSIM of 0.2932, competitive with the leading text-to-video methods of the time. More tellingly, it improves over its own first-stage text-to-video pre-training, which scored FVD 803. Adding compositional, multi-condition control did not cost text-to-video quality; if anything it helped slightly.

Qualitatively, the model follows the conditions it is given. The figure below shows compositional image-to-video generation: a single frame plus a caption already produces a video, and adding one more temporal condition, a depth, mask, or motion-vector sequence, gives finer control over how the structure evolves over time.

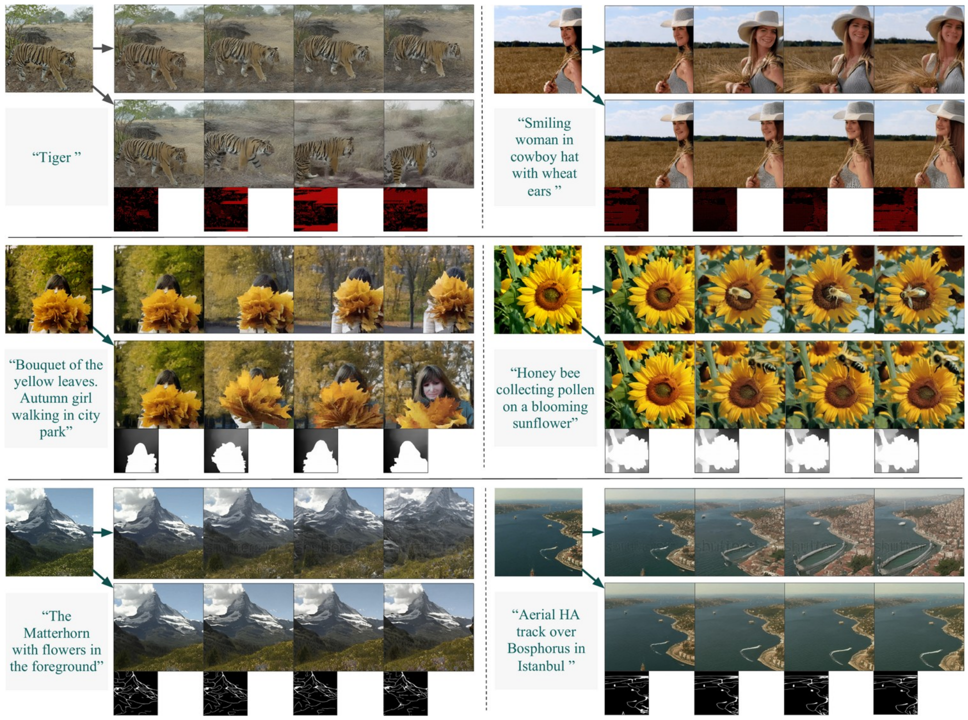

Figure 4. Compositional image-to-video generation. Six examples, each with two rows: the top uses only a single image plus text, the bottom adds one temporal condition. Comparing the two rows in each pair, adding the temporal condition makes the subject's motion and the way its structure evolves track the intent more closely, while the subject's identity stays consistent.

The most vivid demonstration of motion control is directing a video from nothing but a simple hand-drawn stroke. The figure below places VideoComposer next to CogVideo, which offers only coarse motion control: given a direction arrow, or even a curved path, VideoComposer moves a box along the specified direction and walks a tiger along an arc, with fine-grained and flexible control.

Figure 9. Versatile motion control from hand-crafted motions. (a) CogVideo offers only limited motion control; (b) VideoComposer, given a single red hand-drawn stroke as the trajectory, makes "a box move from left to right" and "a tiger walk along an arc on the grassland," with the direction and path clearly controllable. This is what it means for motion vectors plus the STC-encoder to turn motion into a dimension the user can sketch and edit.

Ablation: what the motion vectors and the STC-encoder each contribute

The paper isolates the two components with a motion-control error (lower is better). Text alone gives an error of 4.03. Adding motion vectors as an explicit temporal condition drops it to 2.67, and turning on the STC-encoder brings it down further to 2.18. Both parts matter: the motion vectors supply the temporal signal, and the STC-encoder makes the model actually use it, sharpening motion control and inter-frame consistency.

Table 1. Motion-controllability ablation (motion-control error, lower is better). Motion vectors and the STC-encoder each give a clear reduction, and the two together do best.

Configuration	Motion-control error ↓
Text only	4.03
+ Motion vectors	2.67
+ Motion vectors + STC-encoder (full)	2.18

The qualitative ablation tells the same story: removing the STC-encoder degrades both how well the video follows the specified temporal structure and how consistent it stays across frames. The figure below marks these deficient regions with red boxes.

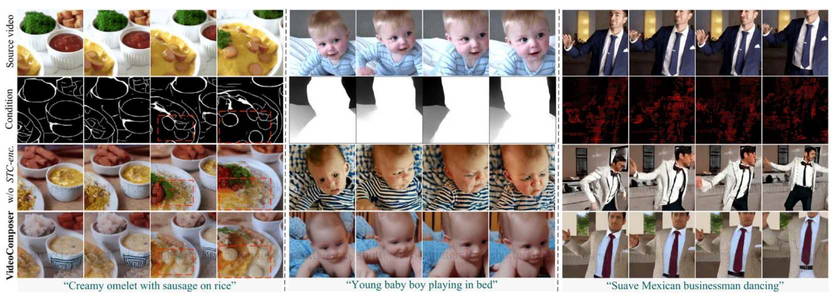

Figure 10. Qualitative ablation. Three representative examples; the last two rows compare VideoComposer without the STC-encoder against the full model under the same text plus a single temporal condition (sketch, depth, or motion vectors). Red boxes mark deficient regions: without the STC-encoder, structural fidelity and inter-frame consistency are visibly weaker.

Takeaway: motion as an editable dimension

The value of VideoComposer is less any single headline number than the framing it offers: treat a video as a composition of textual, spatial, and temporal conditions, give the temporal dimension an explicit signal drawn from compressed-video motion vectors, and pair it with a unified STC-encoder that digests heterogeneous conditions while holding consistency together. It is trained on two public datasets, WebVid10M (about 10.3M video-caption pairs) and LAION-400M, and freely composes 8-plus condition types, from text and images to sketches, motion vectors, and depth, mask, and sketch sequences.

It is worth staying honest about scope. The reported numbers are zero-shot text-to-video metrics and qualitative demonstrations, and motion controllability is a dedicated error on 1000 caption-video pairs, not a general "motion-correctness" score; resolution, length, and realism remain bounded by the underlying VLDM. Still, the direction is clear: decompose a video into conditions, give motion its own explicit signal, and generation becomes markedly more controllable, from text and sketches to depth maps or two hand-drawn strokes.